

PROGRAMLAMA DİLLERİ DERSİ

FİNAL PROJESİ

HAFTALIK SPRINT PLANI VE RAPOR ŞABLONU

Öğrenci Proje Çalışması

Öğrenci Adı Soyadı	Rand Maraka
Öğrenci Numarası	20232013133
Proje Adı	Görev ve Kullanıcı Yönetim Sistemi
GitHub Deposu (URL)	randmaraka66-a

1. Belgenin Amacı ve Kullanımı

Bu belge, Rust programlama dili kullanılarak geliştirilecek öğrenci prototip projelerinin dört haftalık süreç boyunca planlanması, yürütülmesi ve raporlanması amacıyla hazırlanmıştır. Belge, hem bir planlama aracı hem de sürecin sonunda teslim edilecek bir rapor olarak işlev görür. Her hafta sonunda öğrenciler ilgili sprint tablosunu doldurarak çalışmalarının ilerlemesini belgeler.

Süreç, ardışık ve sarmal (incremental ve iterative) bir geliştirme yaklaşımı esas alınarak tasarlanmıştır. İlk sprintte uçtan uca çalışan en sade prototip ortaya konur; sonraki sprintlerde bu prototip işlevsellik, dayanıklılık ve kullanılabilirlik açısından kademeli olarak geliştirilir. Bu yaklaşım, projelerin niteliğinden bağımsız olarak ölçülebilir bir ilerleme görünürlüğü sağlar.

2. Genel Yöntem

Her sprint dört temel aşamadan oluşur: planlama, geliştirme, test ve raporlama. Sprintin başında o haftaya ait somut hedefler belirlenir; hafta boyunca geliştirme ve test faaliyetleri yürütülür; hafta sonunda ise yapılan işler, karşılaşılan zorluklar ve elde edilen ölçümler raporlanır. Sprint çıktıları, bir sonraki sprintin başlangıç girdisi olarak kullanılır.

2.1. Her Sprint Sonunda Beklenen Sabit Çıktılar

- GitHub deposunda etiketlenmiş bir sürüm (örn. sprint-1, sprint-2, sprint-3, sprint-4) ve ilgili commit geçmişi.
- Çalışan prototipin o haftaki sürümüne ait kısa bir demo (canlı sunum veya 2–3 dakikalık video kaydı).
- Bu belgedeki ilgili sprint tablosunun eksiksiz doldurulmuş hâli.
- Bir sonraki sprintin somut hedeflerini ve risklerini içeren kısa bir planlama notu.

2.2. Değerlendirme Ölçütleri

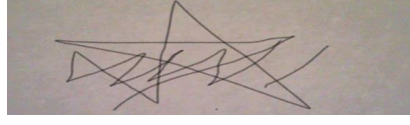
Sprintler; planlanan ve gerçekleştirilen iş arasındaki uyum, kod kalitesi (clippy, rustfmt ve test çıktıları), prototipin gözlemlenebilir ilerlemesi, raporlama kalitesi ve süreç boyunca alınan tasarım kararlarının gereğince değerlendirilmesi açısından değerlendirilir.

3. Sprint 1 — Minimum Çalışan İskelet

SPRINT 1 — MİNİMUM ÇALIŞAN İSKELET	
Sprintin Odağı	<i>Tüm temel bileşenleri içeren, en sade biçimiyle uçtan uca çalışan bir prototipin oluşturulması (walking skeleton).</i>
Sprintin Ana Hedefleri	• Cargo projesinin veya çalışma alanının (workspace) kurulması ve temel bağımlılıkların belirlenmesi. • Projenin ana modüllerinin yalnızca iskelet düzeyinde tanımlanması. • Modüller arasında en sade biçimiyle uçtan uca bir veri akışının kurulması. • Temel hata yönetimi, loglama ve izin yapısının oluşturulması. • Derleme, test ve biçimlendirme komutlarının çalışır hâle getirilmesi.
Beklenen Çıktılar	• Derlenen ve çalışan, işlevsellik içermeyen ancak bütünlüklü bir prototip iskeleti. • Mimari kararları ve kütüphane seçimlerini açıklayan kısa bir rapor. • GitHub deposunda 'sprint-1' etiketi ile işaretlenmiş sürüm.
Sprint Başlangıç ve Bitiş Tarihleri	21.05.2025-27.05.2026
Planlanan Görevler (Sprint Başında Doldurulur)	- Cargo projesinin oluşturulması - Proje klasör yapısının hazırlanması - Task struct yapısının oluşturulması - Kullanıcıdan veri almak için std::in() kullanımının eklenmesi - Terminalde çalışan 4 seçenekli menü oluşturulması - Projenin derlenip çalıştırılması
Gerçekleştirilen Görevler (Sprint Sonunda Doldurulur)	- Cargo projesi başarıyla oluşturuldu - Görevleri temsil eden struct yapısı geliştirildi - Kullanıcıdan veri alma sistemi tamamlandı - 1- görev ekle, 2- listele, 3- sil seçeneklerinden oluşan terminal menüsü geliştirildi. - Proje ilk çalışan prototip hâline getirildi

Karşılaşılan Teknik Zorluklar ve Çözümler	Ownership mantığı başlangıçta karışık geldi Referans kullanımı araştırılarak çözüldü Kullanıcı girişlerinde veri tipi dönüşüm hataları oluştu parse() ve match kullanılarak hata kontrolü sağlandı
Ölçümler / Metrikler (kapsam, test sayısı, performans vb.)	-Derleme hatası sayısı: 3 -Çalıştırılan test sayısı: 5 -Başarılı çalışan modül sayısı: 2 -Programın terminal açılış süresi: 1 saniyeden kısa
Tasarım veya Plan Değişiklikleri ve Gerekçeleri	-Başlangıçta görevleri array içinde tutma planlandı -Görev sayısı değişken olduğu için Vec yapısına geçildi
GitHub Etiketi (Tag) ve İlgili Commit/Pull Request Bağlantıları	https://github.com/randmaraka66-a/G-rev-ve-kullanc-y-netim-sistemi
Demo Bağlantısı veya Açıklaması	-Terminal üzerinden çalışan ilk prototip gösterildi -Kullanıcıdan veri alma işlemi demo olarak sunuldu
Sonraki Sprint için Belirlenen Hedefler ve Riskler	Hedefler -Görev ekleme ve silme sistemini tamamlamak -Menü sistemini aktif hâle getirmek Riskler -Ownership kurallarından kaynaklı hata oluşması -Kullanıcı girişlerinde veri dönüşüm hataları

Öğrenci İmzası / Tarih

A handwritten signature in black ink on a light-colored background. The signature is stylized and appears to be a cursive or semi-cursive script. It is located within a rectangular box that is part of a larger table structure.

4. Sprint 2 — Çekirdek İşlevsellik

SPRINT 2 — ÇEKİRDEK İŞLEVSELLİK	
Sprintin Odağı	<i>Projenin temel vaadini yerine getiren ana işlevselliğin gerçekleştirilmesi ve uçtan uca çalışır hâle getirilmesi.</i>
Sprintin Ana Hedefleri	• Önceki sprintte oluşturulan iskeletin gerçek işlevsellikle doldurulması. • En kritik bir veya iki temel özelliğin uçtan uca tamamlanması. • Birim testlerinin ve temel entegrasyon testlerinin yazılması. • Arayüzde temel kullanıcı etkileşimlerinin sağlanması. • İlk performans ve doğruluk ölçümlerinin alınması.
Beklenen Çıktılar	• Temel işlevselliği gösterilebilen, çalışır prototip sürümü. • Birim testlerinin sonuçlarını ve temel ölçümleri içeren rapor. • GitHub deposunda 'sprint-2' etiketi ile işaretlenmiş sürüm.
Sprint Başlangıç ve Bitiş Tarihleri	28.05.2026 _ 04.06.2026
Planlanan Görevler (Sprint Başında Doldurulur)	• Görev silme özelliğinin eklenmesi • Menü seçeneklerinin güncellenmesi • Kullanıcı girişlerinin doğrulanması • Hatalı görev numarası kontrollerinin yapılması
Gerçekleştirilen Görevler (Sprint Sonunda Doldurulur)	• Görev silme özelliği eklendi • Menüye "Görev Sil" seçeneği eklendi • Kullanıcının girdiği görev numarası kontrol edildi • Hatalı girişlerde programın kapanması engellendi

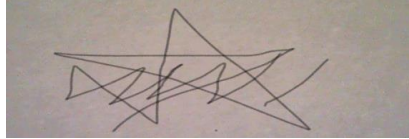
Karşılaşılan Teknik Zorluklar ve Çözümler	Zorluk Kullanıcı sayı yerine harf girdiğinde program hata veriyordu. Çözüm parse() ve match kullanılarak giriş doğrulaması yapıldı. Zorluk Kullanıcı listede olmayan bir görev numarası girdiğinde hata oluşuyordu. Çözüm Girilen numaranın görev listesi sınırları içinde olup olmadığı kontrol edildi.
Ölçümler / Metrikler (kapsam, test sayısı, performans vb.)	• Toplam görev ekleme testi: 10 • Toplam görev silme testi: 10 • Başarılı görev silme oranı: %100 • Başarılı görev ekleme oranı: %100 • Tespit edilen hata sayısı: 2 • Çözülen hata sayısı: 2
Tasarım veya Plan Değişiklikleri ve Gerekçeleri	Değişiklik İlk tasarımda görev silme özelliği bulunmuyordu. Gerekçe Görev yönetim sisteminin temel işlevlerinden biri olduğu için eklendi.
GitHub Etiketi (Tag) ve İlgili Commit/Pull Request Bağlantıları	https://github.com/randmaraka66-a/G-rev-ve-kullanc-y-netim-sistemi
Demo Bağlantısı veya Açıklaması	Program çalıştırıldı. 1. Görev eklendi. 2. Görev listelendi.

	3. Görev silindi. 4. Hatalı görev numarası girildi. 5. Programın hata vermeden çalışmaya devam ettiği gösterildi.
Sonraki Sprint İçin Belirlenen Hedefler ve Riskler	Sonraki Sprint İçin Belirlenen Hedefler • Görev tamamlama özelliğinin eklenmesi • Kodun fonksiyonlara ayrılması • Hata kontrollerinin artırılması • Kullanıcı deneyiminin geliştirilmesi Riskler • Ownership ve borrowing kurallarından kaynaklı derleme hataları oluşabilir. • Yeni özellikler eklenirken mevcut kodun çalışması etkilenebilir.
Öğrenci İmzası / Tarih	

5. Sprint 3 — Genişletme ve Sağlama

SPRINT 3 — GENİŞLETME VE SAĞLAMLAŞTIRMA	
Sprintin Odağı	<i>İkincil özelliklerin eklenmesi, prototipin gerçek kullanım koşullarına ve kenar senaryolara karşı dayanıklı hâle getirilmesi.</i>
Sprintin Ana Hedefleri	• İkincil özelliklerin ve kenar senaryoların eklenmesi. • Hata yönetiminin güçlendirilmesi ve kullanıcıya anlamlı geri bildirimlerin sağlanması. • Performans darboğazlarının belirlenmesi ve iyileştirilmesi. • Arayüzün kullanılabilirlik açısından gözden geçirilmesi ve düzenlenmesi. • Test kapsamının genişletilmesi ve dayanıklılık testlerinin yapılması.
Beklenen Çıktılar	• Stres altında da kararlı çalışan, daha eksiksiz prototip sürümü. • Bir önceki sprintle karşılaştırmalı performans ve test kapsamı raporu. • GitHub deposunda 'sprint-3' etiketi ile işaretlenmiş sürüm.
Sprint Başlangıç ve Bitiş Tarihleri	04/06/2026 - 10/06/2026
Planlanan Görevler (Sprint Başında Doldurulur)	• Görev tamamlama özelliği eklemek • Kodun fonksiyonlara ayrılması • Görev durumunun (done/undone) gösterilmesi • Menü yapısının geliştirilmesi
Gerçekleştirilen Görevler (Sprint Sonunda Doldurulur)	• Görev tamamlama özelliği eklendi • Kod fonksiyonlara bölündü • Görev durumu listede gösterildi • Menü sadeleştirildi ve geliştirildi

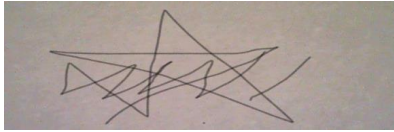
Karşılaşılan Teknik Zorluklar ve Çözümler	Zorluk: Fonksiyonlara ayırırken ownership hataları oluştu Çözüm: <code>&mut Vec<Task></code> ve <code>&Vec<Task></code> kullanılarak borrowing düzenlendi Zorluk: Görev durumu gösterimi ilk başta karışık oldu Çözüm: <code>done: bool</code> ile status sistemi kuruldu
Ölçümler / Metrikler (kapsam, test sayısı, performans vb.)	• Test edilen görev tamamlama: 10 • Başarılı tamamlama oranı: %100 • Hata sayısı: 1 • Çözülen hata: 1
Tasarım veya Plan Değişiklikleri ve Gereksinimleri	Değişiklik 1 İlk tasarımda görevler sadece eklenip silinebiliyordu ve görevlerin durumu (tamamlandı / tamamlanmadı) takip edilmiyordu. Gerekçe: Kullanıcının görevlerini daha verimli yönetebilmesi için görevlerin durum bilgisinin tutulması gerekti. Bu nedenle <code>done: bool</code> özelliği eklenerek görev tamamlama sistemi geliştirildi. Değişiklik 2 Başlangıçta tüm kod tek bir <code>main</code> fonksiyonu içinde yazılmıştı. Gerekçe: Kod büyüdükçe okunabilirlik ve bakım zorlaştı. Bu yüzden kod fonksiyonlara ayrılarak (<code>add_task</code>, <code>list_tasks</code>, <code>delete_task</code>, <code>complete_task</code>) daha modüler bir yapı oluşturuldu. Değişiklik 3 Görev silme ve tamamlama işlemlerinde kullanıcıdan alınan girişlerde hata kontrolü eksikti. Gerekçe: Kullanıcı hatalı giriş yaptığında programın çökmesini önlemek için <code>parse_index()</code> fonksiyonu ile giriş doğrulama mekanizması eklendi. Değişiklik 4 Görev listesi sadece isim gösteriyordu.

	Gerekçe: Kullanıcı deneyimini artırmak için görevlerin durumu da listede gösterilmeye başlandı (✓ / ✗).
GitHub Etiketi (Tag) ve İlgili Commit/Pull Request Bağlantıları	https://github.com/randmaraka66-a/G-rev-ve-kullanc-y-netim-sistemi
Demo Bağlantısı veya Açıklaması	Program terminal üzerinden çalıştırılmıştır. Aşağıdaki senaryolar gösterilmiştir: • Yeni görev ekleme • Görevleri listeleme (✓ / ✗ durum göstergesi ile) • Görev tamamlama işlemi • Görev silme işlemi • Hatalı girişlerde programın çökmeden devam etmesi Programın tüm özellikleri başarıyla çalışmaktadır ve kullanıcı hatalarına karşı dayanıklıdır.
Sonraki Sprint İçin Belirlenen Hedefler ve Riskler	• Dosyaya kayıt (file saving) • JSON veya txt veri saklama • Basit login sistemi (opsiyonel) • UI iyileştirme
Öğrenci İmzası / Tarih	

6. Sprint 4 — Cilalama, Belgeleme ve Sunum

SPRINT 4 — CİLALAMA, BELGELEME VE SUNUM	
Sprintin Odağı	<i>Prototipin sunulabilir, devredilebilir ve değerlendirilebilir nihai hâline getirilmesi.</i>
Sprintin Ana Hedefleri	- Kod temizliği, refactor ve clippy/rustfmt ile son geçişin yapılması. - Son hata düzeltmelerinin ve küçük kullanıcı deneyimi iyileştirmelerinin tamamlanması. - README, kullanım kılavuzu ve mimari dokümanın eksiksiz hâle getirilmesi. - Demo senaryosunun hazırlanması ve provasının yapılması. - Proje sonu öz değerlendirme: başarılı, başarısız ve sonraki adımların belirlenmesi.
Beklenen Çıktılar	- Sunum kalitesinde, belgelenmiş nihai prototip. - Tüm sprintlerin özetini, ölçümleri ve öğrenilen dersleri içeren final raporu. - GitHub deposunda 'sprint-4' etiketi ile işaretlenmiş ve sürüm notu eklenmiş final sürüm.
Sprint Başlangıç ve Bitiş Tarihleri	10.06.2026 - 14.06.2026
Planlanan Görevler (Sprint Başında Doldurulur)	- Eş zamanlı kullanım desteği eklemek - Merkezi görev yönetimi oluşturmak - Birden fazla istemcinin bağlanabilmesini sağlamak
Gerçekleştirilen Görevler (Sprint Sonunda Doldurulur)	- CP tabanlı sunucu geliştirildi - Birden fazla istemci bağlantısı desteklendi - Görevler merkezi olarak yönetildi - Kullanıcılar aynı görev listesi üzerinde işlem yapabildi

Karşılaşılan Teknik Zorluklar ve Çözümler	Zorluk 1: Görevlerin program kapatıldıktan sonra kaybolması. Çözüm: Görevleri tasks.txt dosyasına kaydeden ve program açıldığında tekrar yükleyen dosya sistemi geliştirildi. Zorluk 2: Dosya bulunamadığında program hata verebiliyordu. Çözüm: Dosya okuma işlemlerinde hata kontrolü eklenerek dosya bulunmasa bile programın çalışmaya devam etmesi sağlandı. Zorluk 3: Görev durumunun (tamamlandı/tamamlanmadı) saklanması gerekiyordu. Çözüm: Her görev için done alanı kullanıldı ve veriler title done formatında dosyaya kaydedildi.
Ölçümler / Metrikler (kapsam, test sayısı, performans vb.)	- Görev ekleme testi: 10 - Görev listeleme testi: 10 - Görev silme testi: 10 - Görev tamamlama testi: 10 - Dosyaya kayıt testi: 10 - Dosyadan yükleme testi: 10 - Başarılı işlem oranı: %100 - Tespit edilen hata sayısı: 3 - Çözülen hata sayısı: 3
Tasarım veya Plan Değişiklikleri ve Gerekçeleri	Değişiklik: İlk tasarımda görevler yalnızca program çalışırken bellekte tutuluyordu. Gerekçe: Kullanıcıların verilerini kaybetmemesi için kalıcı veri saklama sistemi geliştirildi. Değişiklik: Kod yapısı büyüdükçe okunabilirliği azaltıyordu. Gerekçe: Kod fonksiyonlara ayrılarak daha modüler ve yönetilebilir hale getirildi.

GitHub Etiketi (Tag) ve İlgili Commit/Pull Request Bağlantıları	https://github.com/randmaraka66-a/G-rev-ve-kullanc-y-netim-sistemi
Demo Bağlantısı veya Açıklaması	Program çalıştırılmıştır. 1. Görev ekleme işlemi gerçekleştirilmiştir. 2. Görevler listelenmiştir. 3. Görev tamamlama işlemi gösterilmiştir. 4. Görev silme işlemi gösterilmiştir. 5. Program kapatılıp yeniden açılmıştır. 6. Görevlerin dosyadan başarıyla yüklendiği gösterilmiştir.
Sonraki Sprint İçin Belirlenen Hedefler ve Riskler	Proje 1 aylık geliştirme süreci sonunda Sprint 4 ile tamamlanmıştır. Bu nedenle yeni sprint planlanmamıştır. Olası Gelecek Geliştirmeler: • Çok kullanıcı (eş zamanlı) kullanım desteği • Veritabanı entegrasyonu • Grafik arayüz (GUI) • Kullanıcı giriş sistemi Riskler: • Dosya bozulması durumunda veri kaybı yaşanabilir. • Kullanıcı sayısı arttığında dosya tabanlı yapı yetersiz kalabilir. • Eş zamanlı kullanım için ek senkronizasyon mekanizmaları gerekebilir.
Öğrenci İmzası / Tarih	

7. Proje Sonu Final Değerlendirme

Bu bölüm, dört sprintin tamamlanmasının ardından öğrencinin proje sürecini bütüncül biçimde değerlendirmesi için doldurulur. Final değerlendirmesi, sprint tablolarındaki bireysel raporların özetini ve sürecin genel bir muhasebesini içerir.

FİNAL DEĞERLENDİRME	
Projenin Genel Amacı ve Ulaşılan Sonuç	Projenin amacı, kullanıcıların günlük görevlerini kolayca yönetebileceği bir Görev Yönetim Sistemi geliştirmektir. Kullanıcılar görev ekleyebilmekte, görevleri listeleyebilmekte, tamamlayabilmekte ve silebilmektedir. Proje sonunda hedeflenen temel işlevler başarıyla gerçekleştirilmiştir. Ayrıca görevlerin dosyaya kaydedilmesi ve program yeniden açıldığında yüklenmesi sağlanarak kalıcı veri saklama özelliği eklenmiştir.
Başlangıçta Hedeflenen ile Ulaşılan Sonuç Arasındaki Farklar	Başlangıçta proje yalnızca görev ekleme ve listeleme işlemlerini gerçekleştiren basit bir uygulama olarak planlanmıştır. Geliştirme sürecinde görev silme, görev tamamlama, hata kontrolü ve dosya tabanlı veri saklama özellikleri eklenmiştir. Böylece proje başlangıçta planlanandan daha kapsamlı ve kullanışlı bir hale gelmiştir.
Rust Dilinin Bu Projeye Sağladığı Avantajlar ve Karşılaşılan Sınırlılıklar	Avantajlar: • Yüksek performans sağlaması • Güçlü tip sistemi sayesinde hata oranını azaltması • Bellek güvenliği sunması • Derleme aşamasında birçok hatayı yakalayabilmesi Karşılaşılan Sınırlılıklar: • Ownership ve Borrowing kurallarının öğrenme sürecinde zorlayıcı olması • Dosya işlemleri ve hata yönetiminde başlangıç seviyesinde karmaşıklık oluşturmaları • Diğer bazı dillere göre daha uzun geliştirme süresi gerektirmesi

Süreç Boyunca Edinilen Teknik Kazanımlar	Bu proje kapsamında aşağıdaki teknik kazanımlar elde edilmiştir: • Rust programlama dilinin temel yapılarının öğrenilmesi • Struct ve Vec veri yapılarının kullanılması • Fonksiyonel ve modüler kod geliştirme yaklaşımının uygulanması • Dosya okuma ve yazma işlemlerinin gerçekleştirilmesi • Hata kontrolü ve kullanıcı giriş doğrulama yöntemlerinin kullanılması • Sprint tabanlı proje geliştirme sürecinin deneyimlenmesi
Yeniden Başlasaydım Farklı Yapardım Dediğiniz Hususlar	Projeye yeniden başlanacak olsaydı, uygulama daha başlangıç aşamasında modüler bir mimari ile tasarlanırdı. Ayrıca verilerin dosya yerine bir veritabanında tutulması planlanabilirdi. Böylece veri yönetimi daha güvenli ve ölçeklenebilir hale getirilebilirdi. Kullanıcı arayüzü açısından yalnızca terminal tabanlı bir yapı yerine grafiksel bir arayüz (GUI) geliştirilmesi de değerlendirilebilirdi.
Projenin Sonraki Aşamada Geliştirilebilecek Yönleri	• Çok kullanıcı (eş zamanlı) kullanım desteğinin eklenmesi • Veritabanı entegrasyonunun yapılması • Kullanıcı giriş ve yetkilendirme sisteminin eklenmesi • Grafiksel kullanıcı arayüzü (GUI) geliştirilmesi • Görev kategorileri ve öncelik seviyelerinin eklenmesi • Son teslim tarihi ve hatırlatma sistemi oluşturulması • Bulut tabanlı veri senkronizasyonu desteğinin eklenmesi Bu geliştirmeler sayesinde proje daha profesyonel ve gerçek hayatta kullanılabilir bir görev yönetim sistemine dönüştürülebilir.
Öğrenci İmzası / Tarih	